LABORATORY OF ADVANCED PROGRAMMING – MARCH 2026

WELCOME TO MARS. PLEASE DON'T DIE.

Presented by:

- Alessandro Arellano Altuna
- Lorenzo Bertucci
- Aldo Bosco
- Tiziano Coluzzi

PROJECT GOAL

The goal of this project is to develop an event-driven IoT automation platform for a simulated Mars habitat.

Heterogeneous Ingestion

Collect data from REST devices via polling and from telemetry topics via asynchronous streams.

Internal Normalization

Convert diverse sensor payloads into a unified internal event format for system-wide compatibility.

Event-Driven Logic

Utilize a message broker to decouple data sources from automation processing and the dashboard.

Real-Time Automation

Dynamically evaluate IF-THEN rules to trigger actuator updates instantly upon event arrival.

Persistent Storage

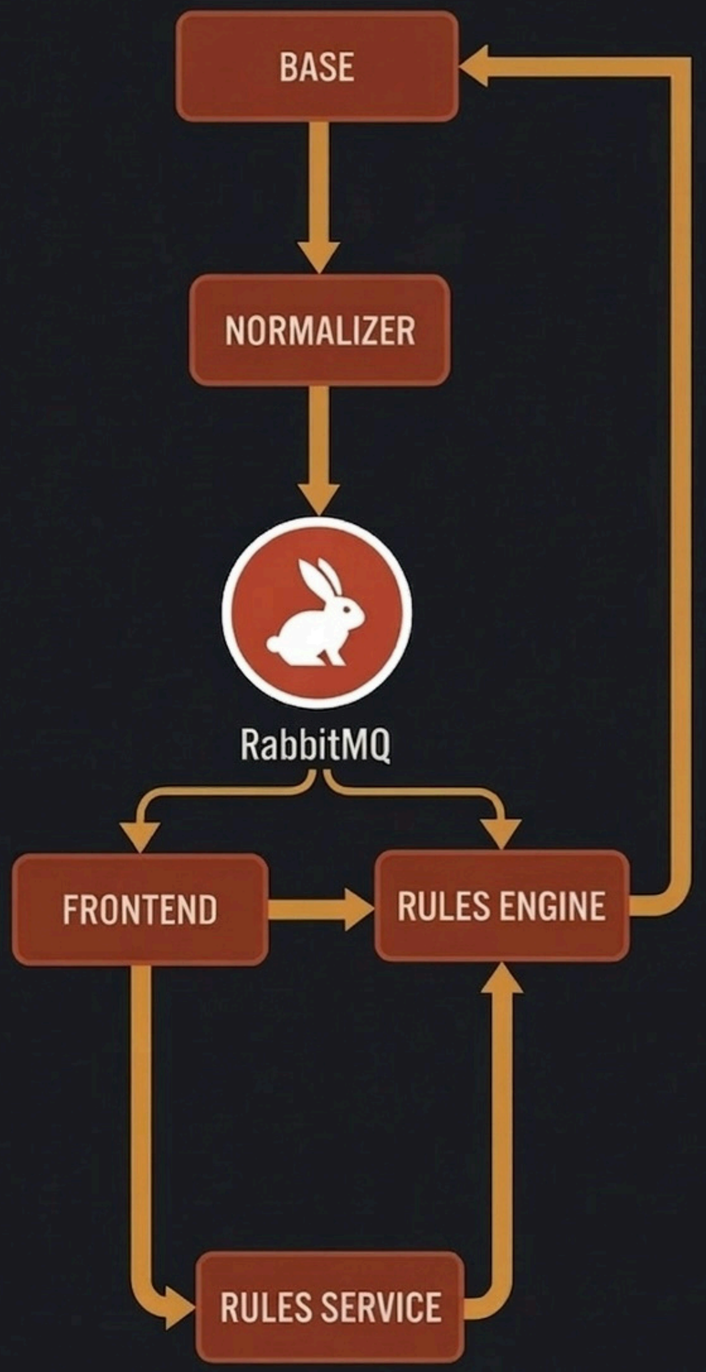
Ensure automation rules survive service restarts while maintaining a real-time sensor state cache.

EVENT-DRIVEN ARCHITECTURE

A distributed microservice architecture designed for scalability and fault tolerance

Main components:

- Normalizer service
- RabbitMQ message broker
- Rule Engine
- Rules Service + PostgreSQL
- Frontend dashboard



NORMALIZER SERVICE

Responsible for collecting and standardizing data from Mars habitat heterogeneous sources.

Hybrid Ingestion

The service polls REST sensors every 10 seconds and listens to asynchronous WebSocket telemetry streams.

Data Normalization

Converts raw, incompatible payloads into a unified *CommonTelemetry* schema using Pydantic models.

Event Publication

Standardized events are published as persistent messages to RabbitMQ, decoupling ingestion from logic.

RULE ENGINE

The system's primary consumer, evaluating automation logic in real-time.

Logic Evaluation

Evaluates dynamic "IF-THEN" conditions (e.g., Temperature > 28°C) as soon as events arrive from the broker

Actuator Control

Triggers automated state updates by issuing REST commands to habitat hardware like fans and heaters.

Control Modes

Implements a state manager supporting AUTO, ON, and OFF modes for manual operator overrides.

RULES SERVICE

A dedicated persistence layer ensuring automation rules survive system restarts.

PostgreSQL Storage

Chosen for ACID compliance and reliable handling of concurrent requests from multiple microservices

RESTful Management

Provides full CRUD (Create, Read, Update, Delete) endpoints for dynamic rule configuration.

State Separation

Maintains persistent rules in the database while keeping live sensor states in-memory for maximum speed.

FRONTEND DASHBOARD

A real-time interface for habitat monitoring and mission-critical control

Real-time Telemetry

Displays live sensor readings and historical trends updated via persistent RabbitMQ consumers

Rule Management UI

Provides a user-friendly interface to manage 20 user stories, from life support to power safety.

Analytics & Rule Management

Combines live charts for trend monitoring with a dedicated panel for real-time CRUD operations on automation rules